# Machine Learning Progress Update

By Ben, Brian, Eric, Kai

# Project Overview

Goal: recognize spoken commands (yes, no, stop, go) using machine learning

Audio signals are converted into mel spectrogram images

A Convolutional Neural Network (CNN) is trained to classify commands

Implemented in MATLAB using the Google Speech Commands dataset

# Dataset Overview

- Used the Google Speech Commands Dataset v0.02 containing short spoken word recordings
- Each clip is 1 second long with a 16 kHz sampling rate
- Commands used in previous stage of the project: yes, no, stop, go
- Dataset was split into training (70%), validation(15%), and testing (15%)
  - Validation added for performance monitoring during training, keeps the test set untouched for a more honest final accuracy.

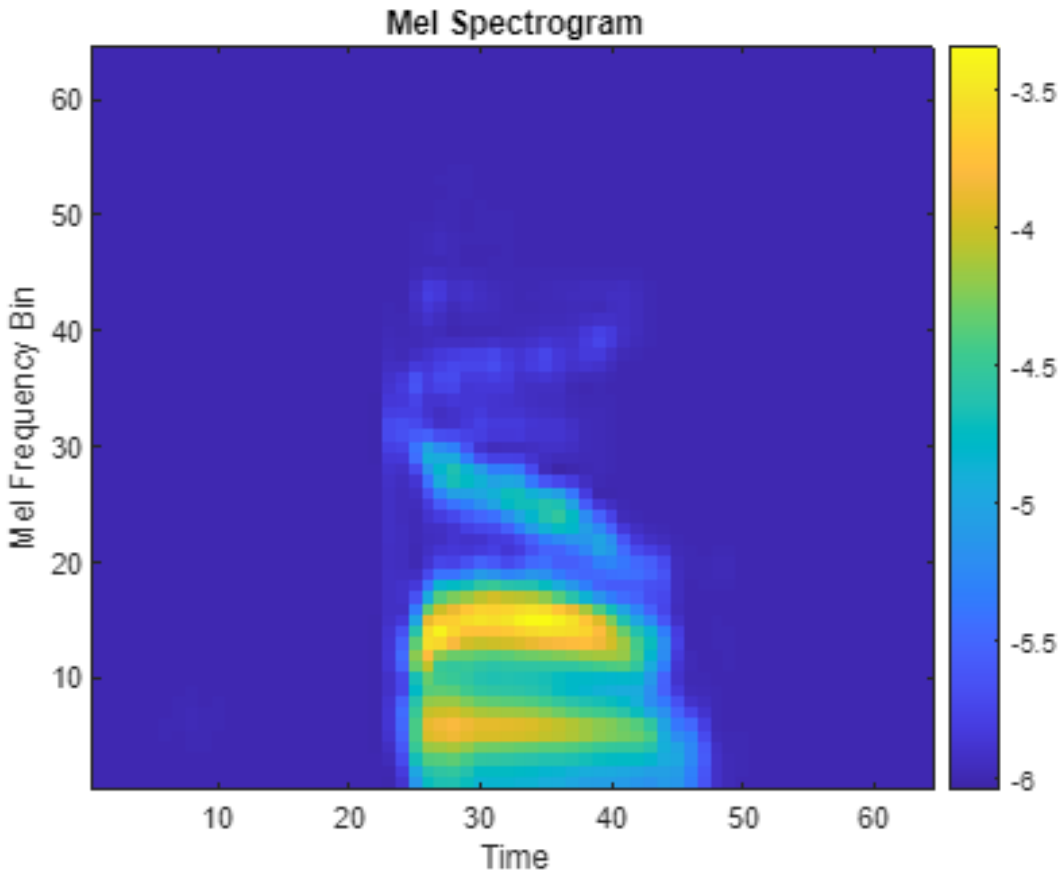



*Figure 1: Mel Spectrograph Result*

# How does the project work?

- Raw audio signals are converted into mel spectrograms
  - Mel spectrograms are used to find how loud a noise is
    - Comparing the noise frequencies of a dataset to words the model determines if it is similar enough to be the same word
- Log scaling is applied to emphasize important speech features
- Each spectrogram is resized to **64 × 64** to serve as CNN input

# Additional Previous Work

- CNN only had 2 layers on convolution
  - Found accuracy ~94% after 2nd convolution
- Previous future work included:
  - Add a third convolution layer
  - Add noise for better accuracy
  - Identify which user is speaking
  - Adding more commands

# New features

- Epochs updated from 10 to 15
- Added dropout layer
- Added 3rd convolution layer
- Added noise to CNN models testing
- Model ensures word is accurate enough to be recognized as a command
- Added “unknown” command for unsure results
- Created SVM model to prove CNN is a superior model
- Development of model that can now identify who is speaking based on a small sample set

# SVM Model

**How it works:**

- W/ audio signal, compute Mel Frequency Cepstral Coefficients (MFCC) of audio.
    - Captures pitch, tone, pronunciation
- From this data, we use mean and standard deviation for features to put into SVM model.

**Conclusions:**

- 75% accuracy using same data set.
- Often confuses “no” and “go”
    - A primary goal is to train the model to be able to detect difference in similar words such as “no” and “go”
- Model is less accurate at each word compared to CNN model

Test Accuracy: 74.80%

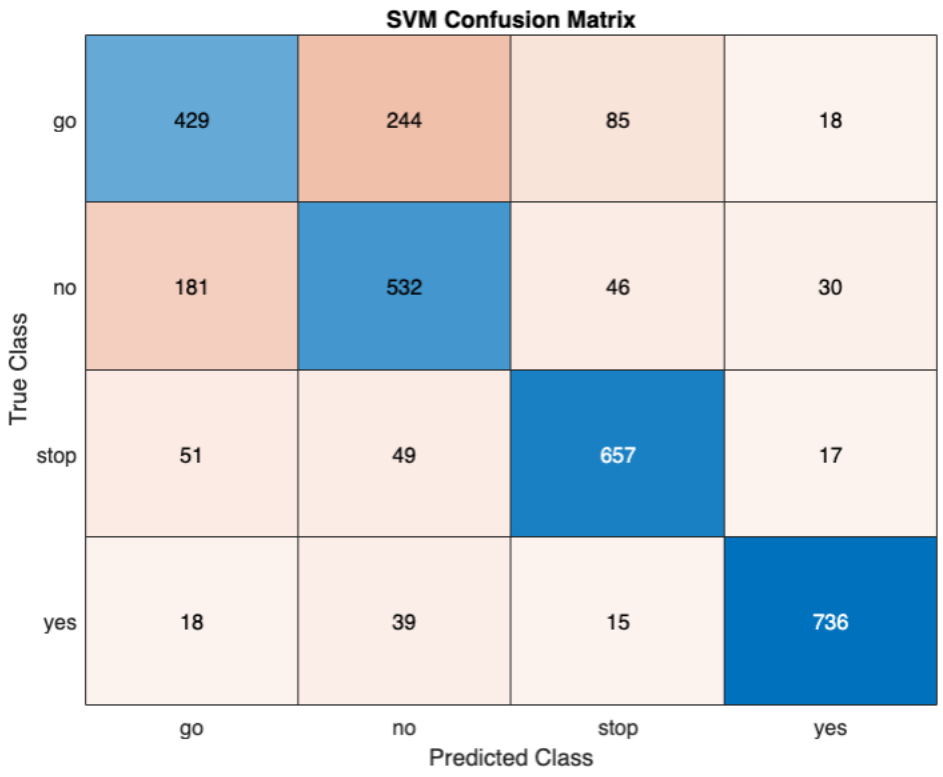



*Figure 2: SVM Confusion Matrix*

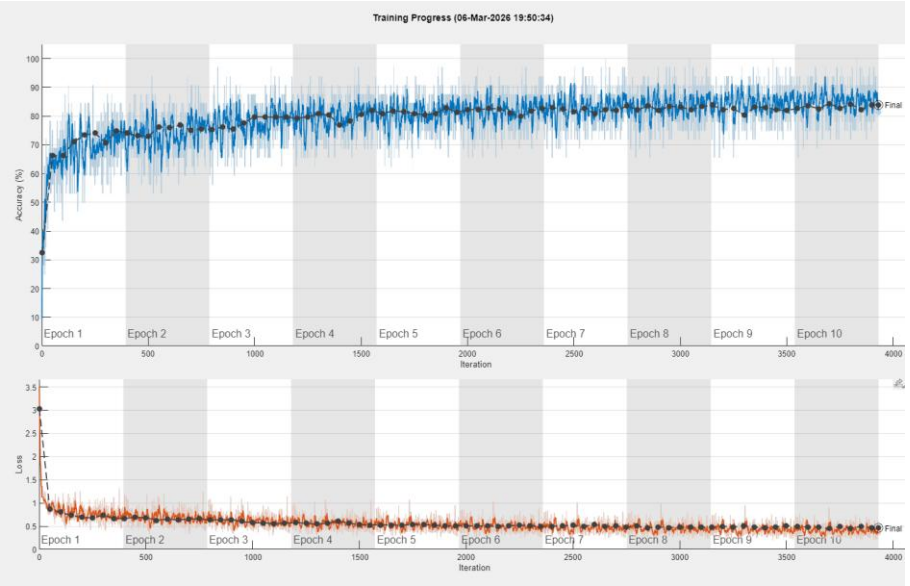



*Figure 3: Results after 1st Convolution*

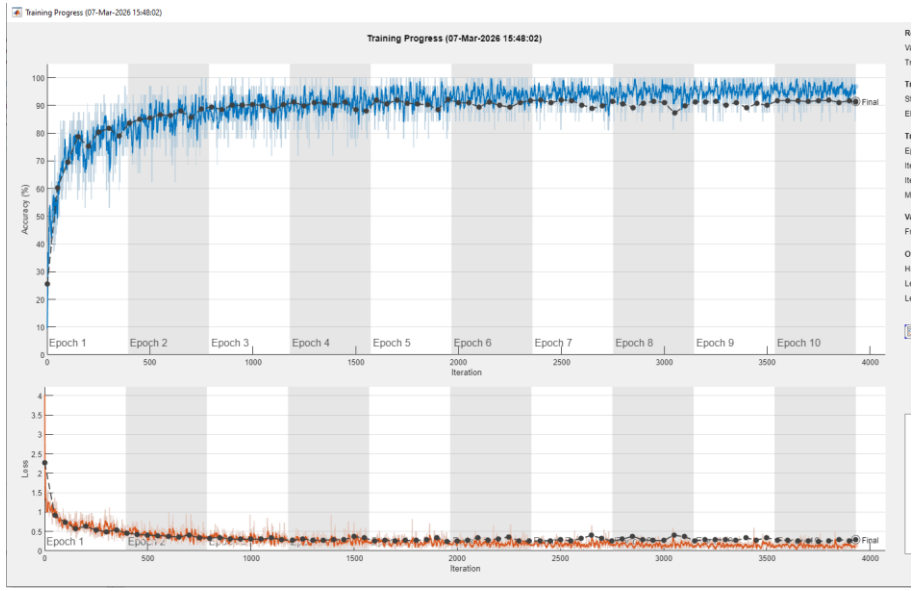



*Figure 4: Results after 2nd Convolution*

# Initial CNN Results

- Accuracy
  - First training ~84%
  - Second training ~92%
    - After adding a second convolution layer accuracy increased by 8%
- Epoch (10)
  - Each training runs through datasets 10 times
- Losses
  - Inaccurate classifications during training (lower number is better)

# CNN Model New Results

*Figure 5: New results after adding noise and 3rd convolution*

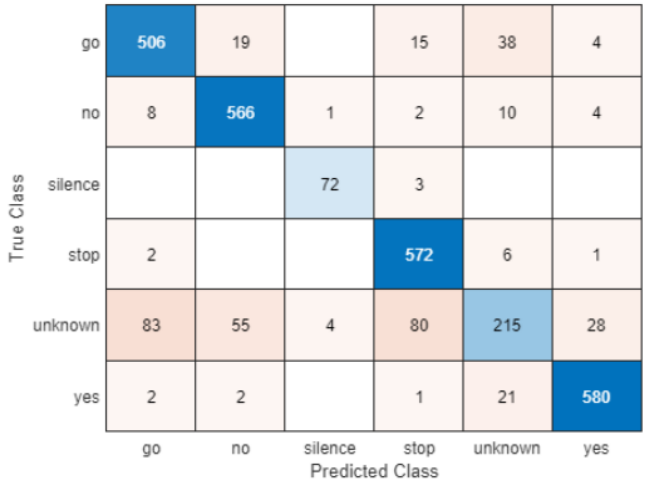



*Figure 6: New Model Confusion Matrix*

- Newest model reports 92% accuracy
  - Similar accuracy even after adding noise
  - Improvement of 7% from second convolution layer
- Model now trains with noise
- Had to add more variance to training data and upgrade our layers to counteract this
  - Third convolution layer added
  - Epochs have been changed from 10 to 15
  - Random time shift and gain to audio files
- “Unknown” category allows accurate results to still be used while getting rid of inaccurate ones

# New code for noise and data variance:

```
%add background noise non-silence samples at random times
if label ~= "silence" && rand < 0.5

    silenceIndex = randi(numel(silenceADS.Files));
    [noiseChunk, noiseFs] = audioread(silenceADS.Files{silenceIndex});

   % if noise sample rate is different, match it to the main audio
    if noiseFs ~= fs
        noiseChunk = resample(noiseChunk, fs, noiseFs);
    end

    % if noise has more than one channel, make it one channel
    if size(noiseChunk,2) > 1
        noiseChunk = mean(noiseChunk,2);
    end

    % if noise is too short, add zeros
    % otherwise, cut it to the right length
    if length(noiseChunk) < targetLength
        noiseChunk = [noiseChunk; zeros(targetLength - length(noiseChunk), 1)];
    else
        noiseChunk = noiseChunk(1:targetLength);
    end

    %adds the noise
    noiseScale = 0.02 + 0.08*rand;
    audioUsed = audioUsed + noiseScale * noiseChunk;

end
%renormalizes after augmentation so the final audio stays in a consistent range
audioUsed = audioUsed ./ (max(abs(audioUsed)) + 1e-6);
```

```
%makes every audio clip exactly 1 second long
targetLength = fs;
if length(audio) < targetLength
    audio = [audio; zeros(targetLength - length(audio), 1)];
else
    audio = audio(1:targetLength);
end

%normalizes the audio so volume differences do not affect training as much
audio = audio ./ (max(abs(audio)) + 1e-6);
audioUsed = audio;

label = string(adsTrain.Labels(i));

% random time shift for all non-silence classes
if label ~= "silence" && rand < 0.5
    shift = randi([-1600, 1600]);
    if shift > 0
        audioUsed = [zeros(shift,1); audioUsed(1:end-shift)];
    elseif shift < 0
        s = abs(shift);
        audioUsed = [audioUsed(s+1:end); zeros(s,1)];
    end
end

%add random gain
if label ~= "silence" && rand < 0.5
    gain = 0.8 + 0.4*rand;
    audioUsed = gain * audioUsed;
end
```

```
layers = [
    %tells MATLAB each input is a 64x64 grayscale image
    imageInputLayer([64 64 1])

    %first convolution layer uses 16 filters to detect simple spectrogram features
    convolution2dLayer(3,16,Padding="same")
    batchNormalizationLayer

    %adds nonlinearity so the network can learn more complex patterns
    reluLayer

    %reduces the image size while keeping the strongest features
    maxPooling2dLayer(2,Stride=2)

    %second convolution layer learns more detailed patterns using 32 filters
    convolution2dLayer(3,32,Padding="same")
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,Stride=2)

    %third convolution layer learns higher-level features using 64 filters
    convolution2dLayer(3,64,Padding="same")
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,Stride=2)

    %drops some neurons during training to help reduce overfitting
    dropoutLayer(0.4)

    %fully connected layer combines extracted features into 64 learned outputs
    fullyConnectedLayer(64)

    %adds nonlinearity before the final classification layer
    reluLayer

    %another dropout layer helps prevent overfitting before classification
    dropoutLayer(0.3)

    %maps the learned features to your command classes
    fullyConnectedLayer(numel(categories(YTrain)))

    %turns outputs into class probabilities
    softmaxLayer

    %computes the classification loss during training
    classificationLayer
];
```

# Voice Recognition: Data & Processing

**Data:**
- 25 audio samples recorded from each team member saying, "U Albany."
- Sample rate: 16khz
- Duration: 2 seconds

**Normalization:**
- Scales signal to range **[-1, 1]**
- Removes loudness differences

**Framing:**
- 25 ms frame size
- 10 ms overlap (hop size)

**Windowing/Hamming:**
- Smooths edges of each frame
- Reduces FFT distortion (spectral leakage)

**Frequency Analysis:**
- FFT converts signal from:
- Time domain to Frequency domain

**Extract features:**
- Signal power
- Average Frequency
- Frequency Variation
- Spectral Max

# Voice Recognition: Model (GMM)

**What is a Gaussian Mixture Model (GMM) ?**

- A Probabilistic model that represents data as a combination of Gaussian (normal) distributions.

**Why Gaussian Mixture Model (GMM)?:**

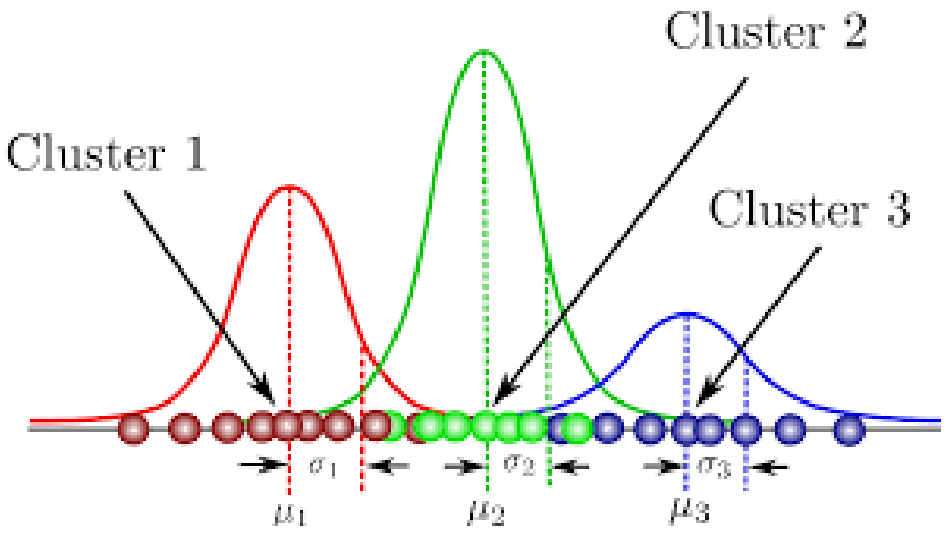
- Tone changes
- Pitch changes
- Variation in speed

$P(x) = \Sigma\ (wk * N(x \mid \mu k, \sigma k))$

- $wk$ = weight
- $\mu k$ = mean of cluster
- $\sigma k$ = Variance of cluster

**Implementation:**

- **Features are extracted from audio and represented as data points:**{Energy, Spectral Mean, Spectral Std, Spectral Max}
- **These data points are used to train a GMM for each speaker:**{Brian, Ben, Eric, Kai}
- **Each GMM models the speaker's voice as a mixture of clusters** (Gaussian distributions)
- **A new sample is then recorded, processed, and converted into feature vectors.**
- **The speaker whose model is most like the features is selected as the predicted speaker**



# Live Demonstration

# Conclusions:

- Developed a speech command recognition system using MATLAB and a Convolutional Neural Network (CNN).
  - Confirmed best model to use based on SVM testing.
- Converted audio recordings into Mel spectrograms, allowing the model to learn patterns in speech as image data.
- Trained the network to classify four commands: “yes,” “no,” “stop,” and “go.”
  - Adding “Unknown” and “Silence” to help accuracy among main four commands (threshold 75%)
- Implemented real-time prediction by recording a new audio sample and classifying which command it is with the trained model.
- Key Takeaway: Transforming audio into spectrogram images is the best way to train a model to effectively learn and recognize spoken commands.
- Developed a voice recognition system using a GMM model.

# Questions?